



ROS2 视觉与控制

为机器装上眼睛



黑马程序员
www.itheima.com

传智教育旗下
高端IT教育品牌



目录

Contents

- ◆ OpenCV&AI登堂入室
- ◆ ROS的视觉管理
- ◆ ROS的坐标转换
- ◆ 正解与反解



OpenCV&AI登堂入室

OpenCV 环境搭建

1. OpenCV安装

```
pip install opencv-python
```

2. 验证安装

2.1

```
python
```

2.2

```
import cv2  
print(cv2.__version__)  
exti()
```

2.3

显示版本号说明成功



数字图像的本质

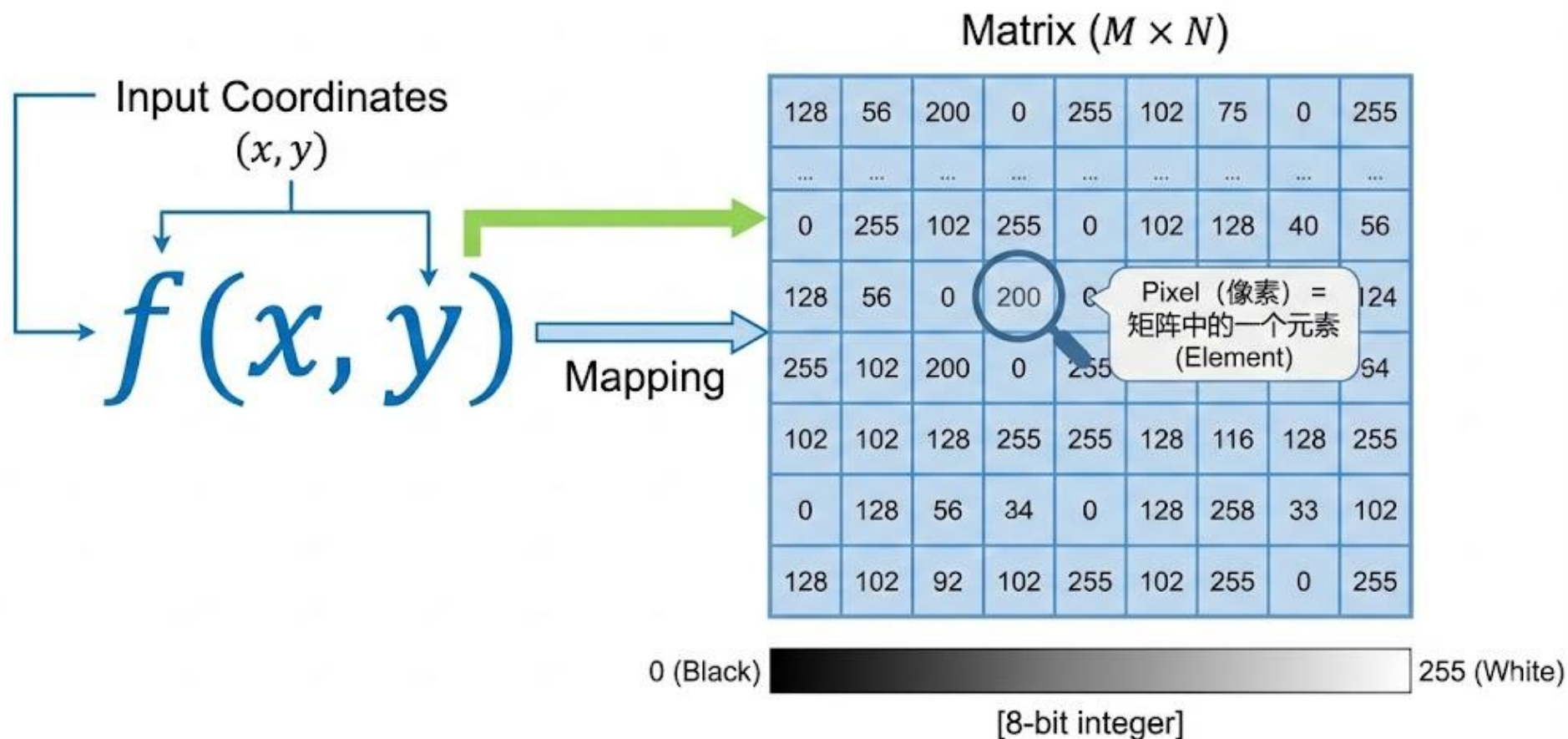
从人类视觉到机器视觉



机器视角 (Machine Vision)

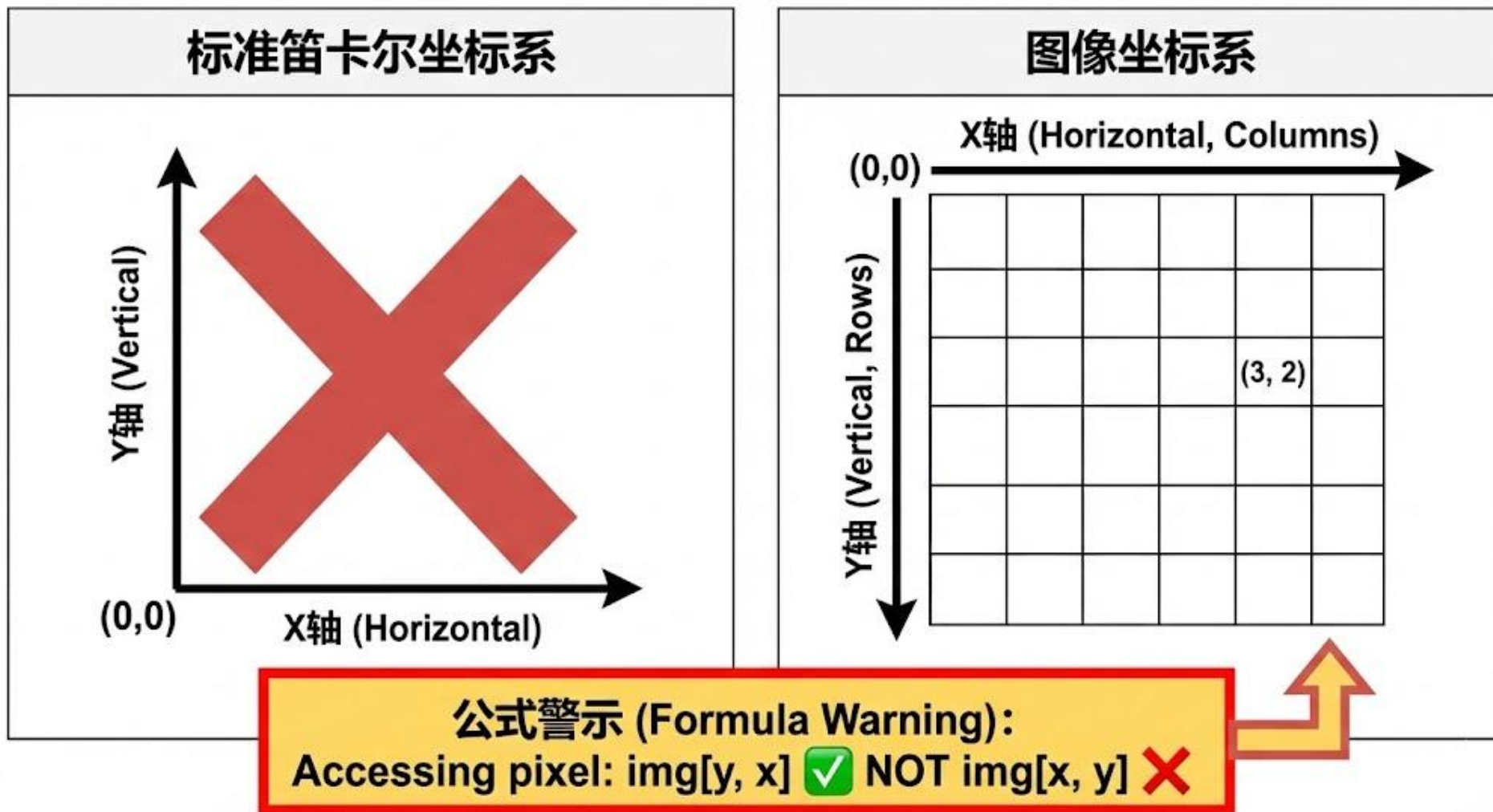
数字图像的本质

Image = Matrix (图像 = 矩阵)



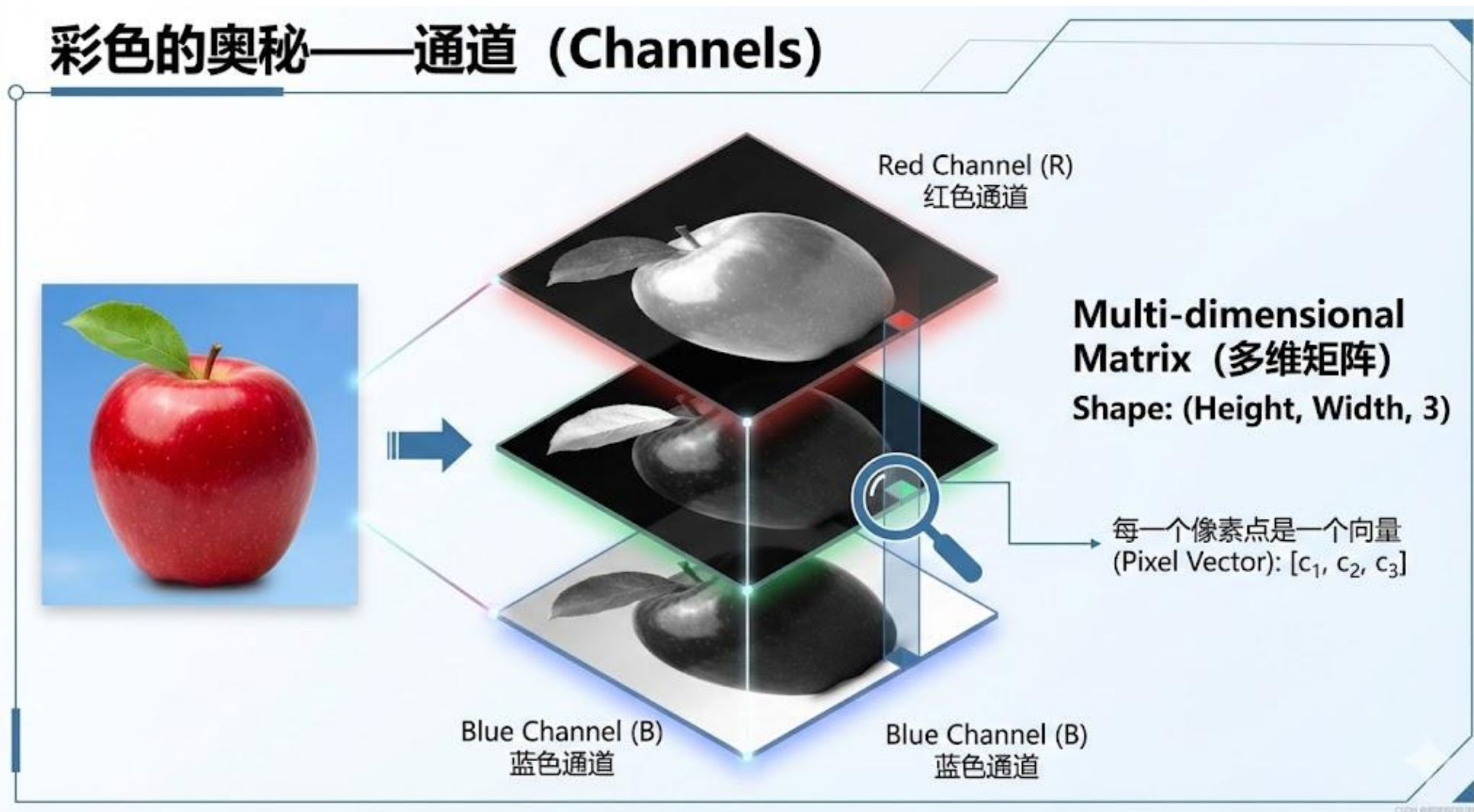
数字图像的本质

原点在哪里



数字图像的本质

彩色的奥秘——通道 (Channels)



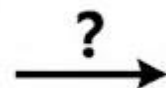
数字图像的本质

OpenCV的历史包袱——BGR vs RGB

BGR (OpenCV Default) vs RGB (Standard)

代码纯推

RGB (标准)

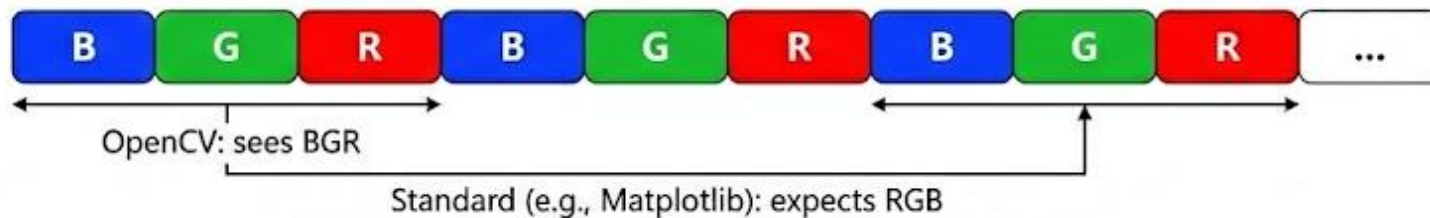


BGR (OpenCV默认读取)



BGR被误读为RGB (BGR interpreted as RGB)

内存示意图
(Memory Layout)



代码Snippet
(Code Snippet)

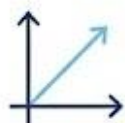
```
import cv2
# 读取图像
img_bgr = cv2.imread('red_car.jpg')
# 转换为RGB
img_rgb = cv2.cvtColor(img_bgr, cv2.COLOR_BGR2RGB) # cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
# 现在可以正确显示
# plt.imshow(img_rgb)
```



计算机视觉基础：关键点回顾



视觉到数据 (Vision to Data): Image is a Matrix ($H \times W \times C$).



坐标系原点 (Origin): Top-Left (0,0); Y-axis points down.



像素访问 (Access): Access: `img[y, x]` (Row, Col).



通道顺序 (Channel Order): OpenCV uses BGR, not RGB.



思考题 (Next Step: A Thought Question):

如果我把一张图的G通道全部置为0，图片会变成什么色调?

数字图像的本质

需求一

1. 创建一张图片
2. 修改某个区域的颜色
3. 查看最终的效果

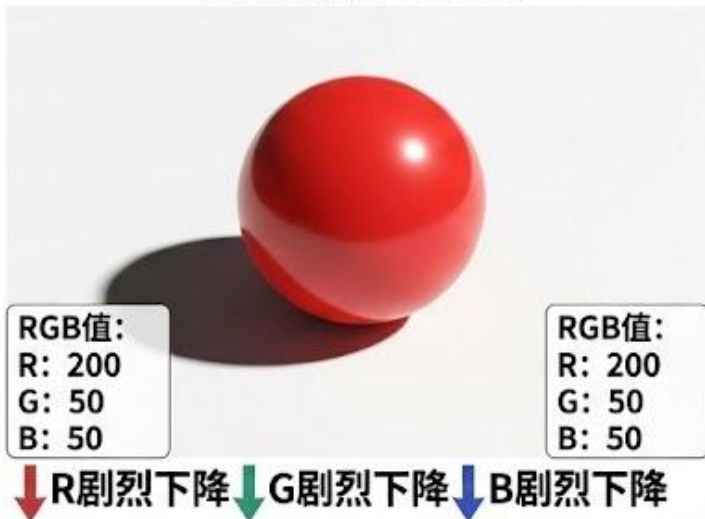
需求二

1. 读取摄像头数据
2. 修改某个区域的颜色
3. 查看最终的效果

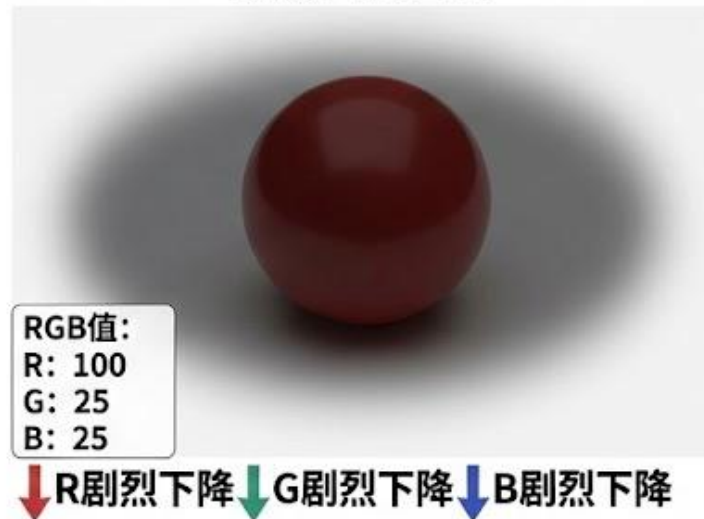
颜色几何学

光照的恶作剧 为什么RGB无法精准追踪?

明亮阳光下的红球



阴影里的红球

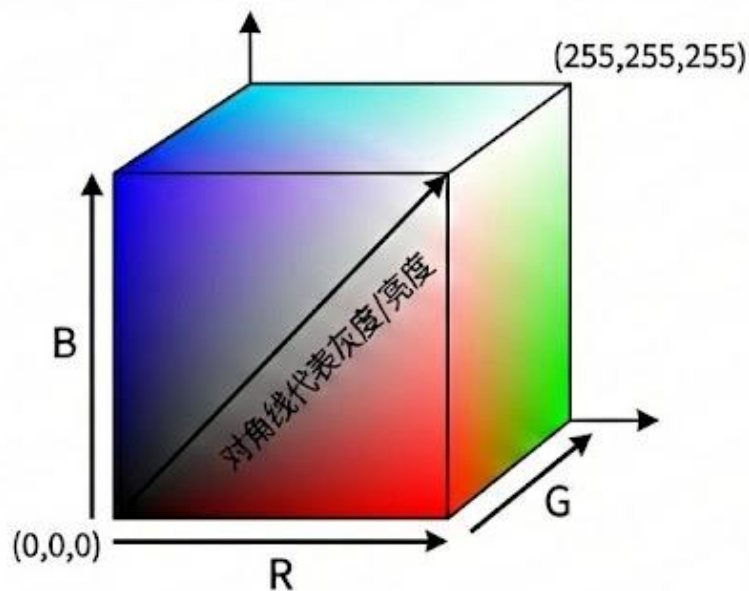


核心问题：“同一个物体，只是光变暗了，为什么它的RGB坐标全变了？”

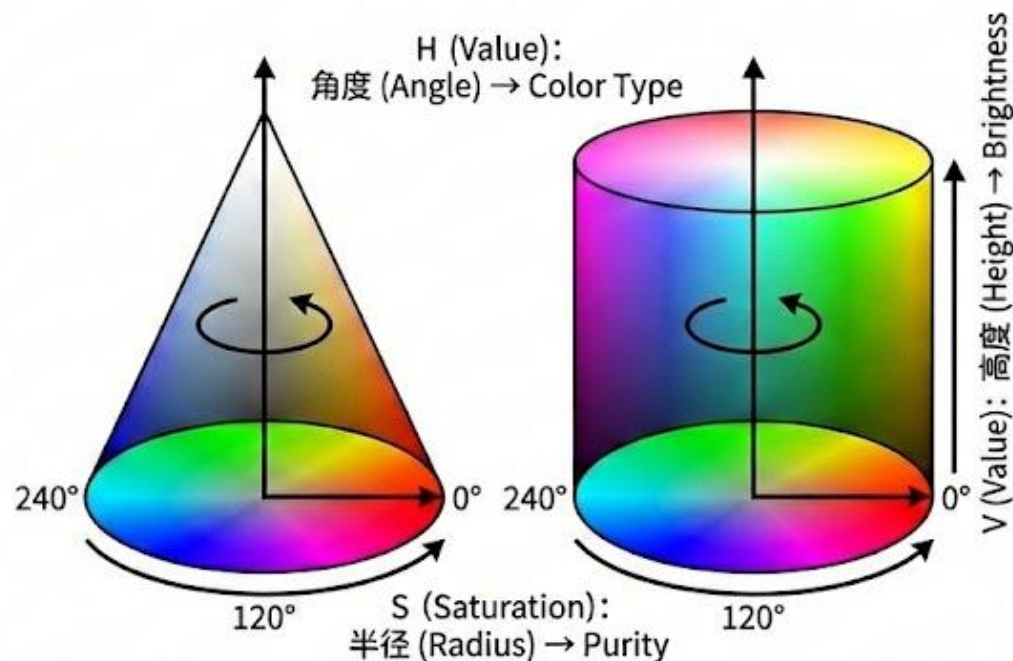
颜色几何学

换一种几何视角 (HSV)

HSV的几何学：圆柱与圆锥



图A: RGB 立方体 (Cube)

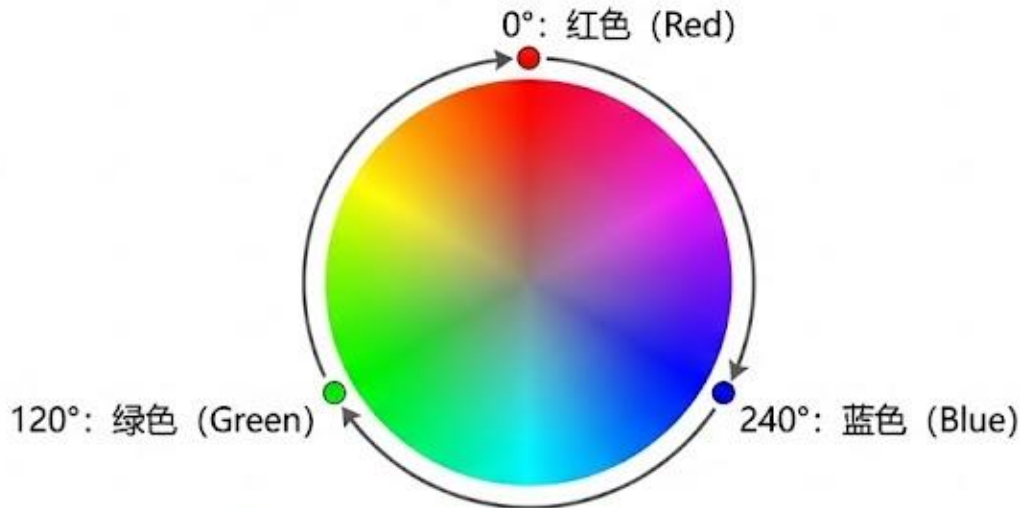


图B: HSV 圆锥体/圆柱体 (Cone/Cylinder)

颜色几何学

颜色的“身份证”—Hue（色相）

核心概念：Hue is the Identity.



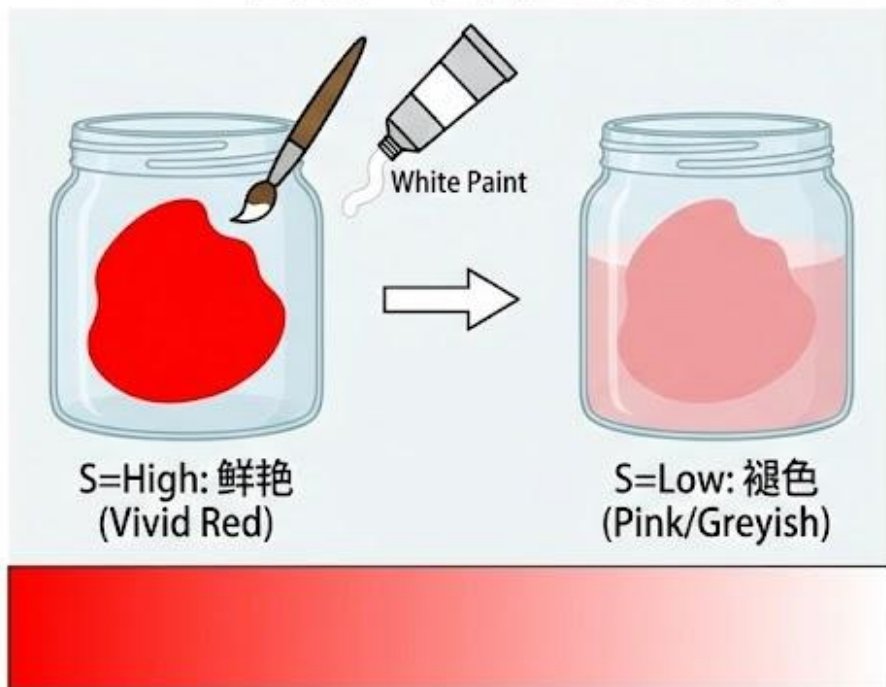
OpenCV特有的坑（警示标志）

数学定义 (Mathematical Definition) : 0 ~ 360°	OpenCV uint8 (OpenCV Implementation) : 0 ~ 180
$OpenCV_Hue = Mathematical_Hue / 2$	

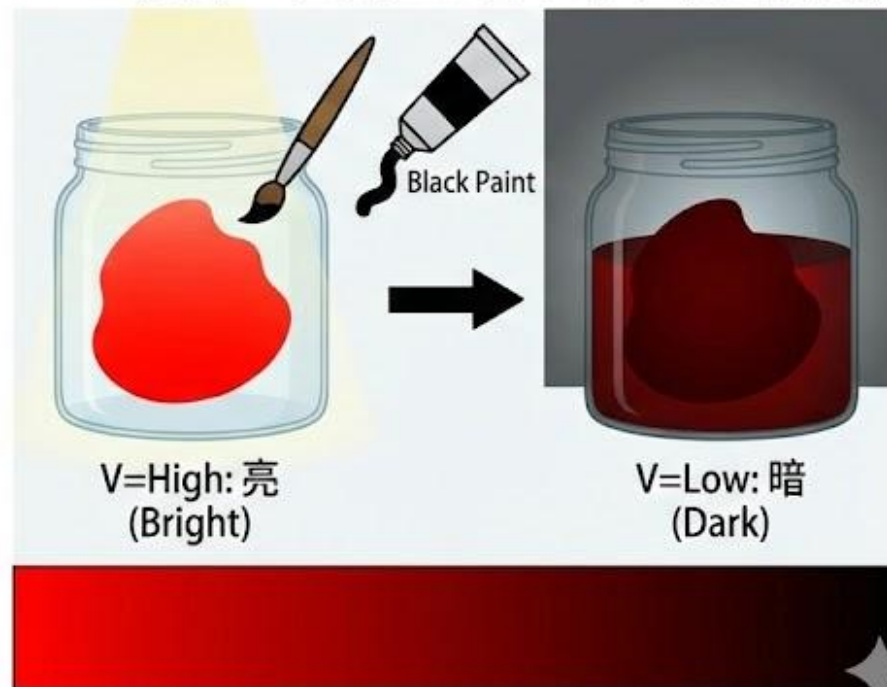
颜色几何学

Saturation & Value: The Mixing Palette

Saturation (饱和度)：往颜料里加了多少白漆？



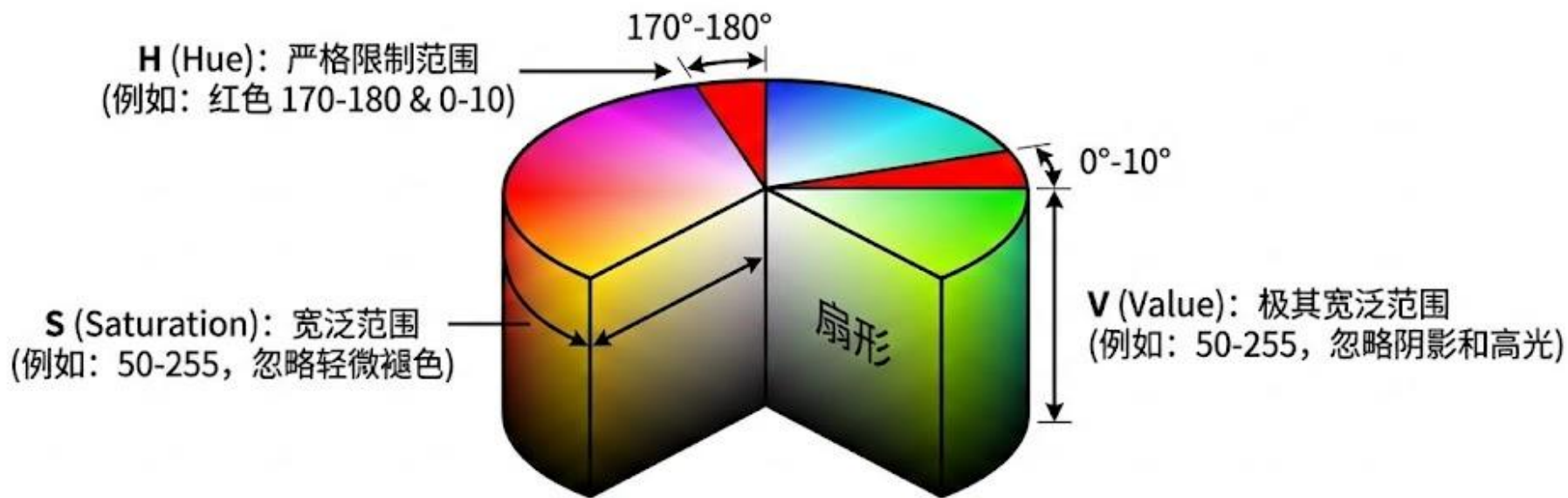
Value (亮度)：往颜料里加了多少黑漆（或光照强弱）？



颜色几何学

鲁棒性追踪的核心—只锁 H

鲁棒追踪的关键



```
cv2.inRange(hsv, lower_red, upper_red)
```


颜色几何学

计算机视觉：色彩空间与追踪要点 (Computer Vision: Color Space & Tracking Takeaways)

1. RGB 混淆



**1. RGB 混淆
(RGB Confusion)**
RGB mixes Color & Light
→ Bad for detection
(RGB混合色彩与光照，
不利于检测)

2. HSV 分离



**2. HSV 分离
(HSV Separation)**
HSV separates Color (H),
Purity (S), Intensity (V)
(HSV分离色相、纯度和亮度)

3. 关键规则



**3. 关键规则
(Critical Rule)**
In OpenCV, $H \in [0, 180]$
(在OpenCV中，H范围为0
到180)

4. 追踪策略



**4. 追踪策略
(Tracking Strategy)**
Strict on H Lenient on V
(严格的H) (宽松的V)



Next Step: 实战演练

编写一个不受阴影干扰的颜色追踪器



颜色几何学

色彩分离实验

1. 读取摄像头数据
2. 将图像由BGR转为HSV
3. 分离HSV
4. 查看分离的结果

掩膜与形态学

什么是掩膜 (Mask) — 从灰度到二值

Thresholding: Creating the Binary Mask

Input: HSV Color Image
(复杂的彩色图)



HSV Image

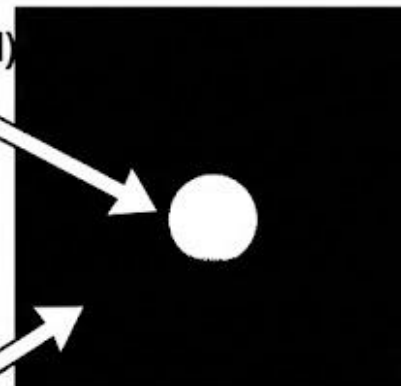
Operation:
`cv2.inRange(lower, upper)`



`cv2.inRange(lower_red, upper_red)`
Thresholding / Segmentation

Selects pixels within specified HSV range

Output: Binary Image
(黑白分明的图)



Binary Mask

White (255):
Region of Interest (ROI)
-> "The Object"

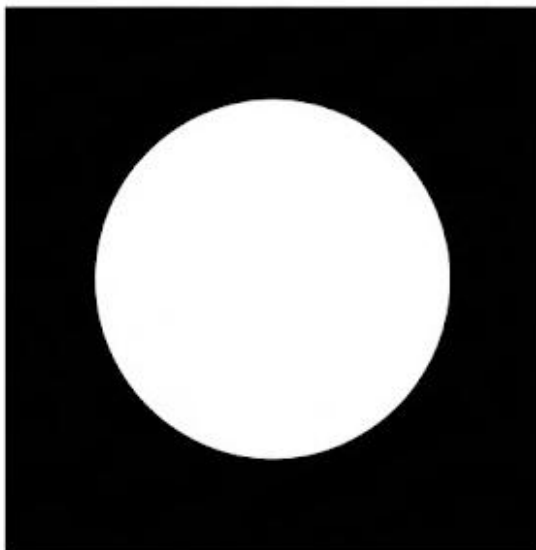
Black (0):
Background ->
"Ignored"

掩膜与形态学

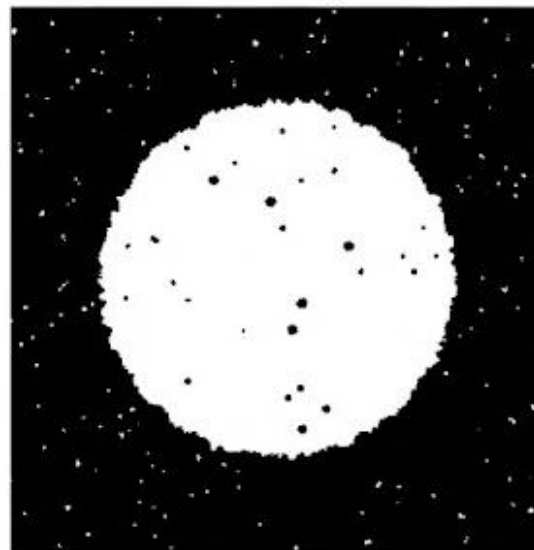
现实很骨感—噪声 (Noise)

The Reality: Salt & Pepper Noise

理想的Mask (Ideal Mask)



实际的Mask (Real Mask)



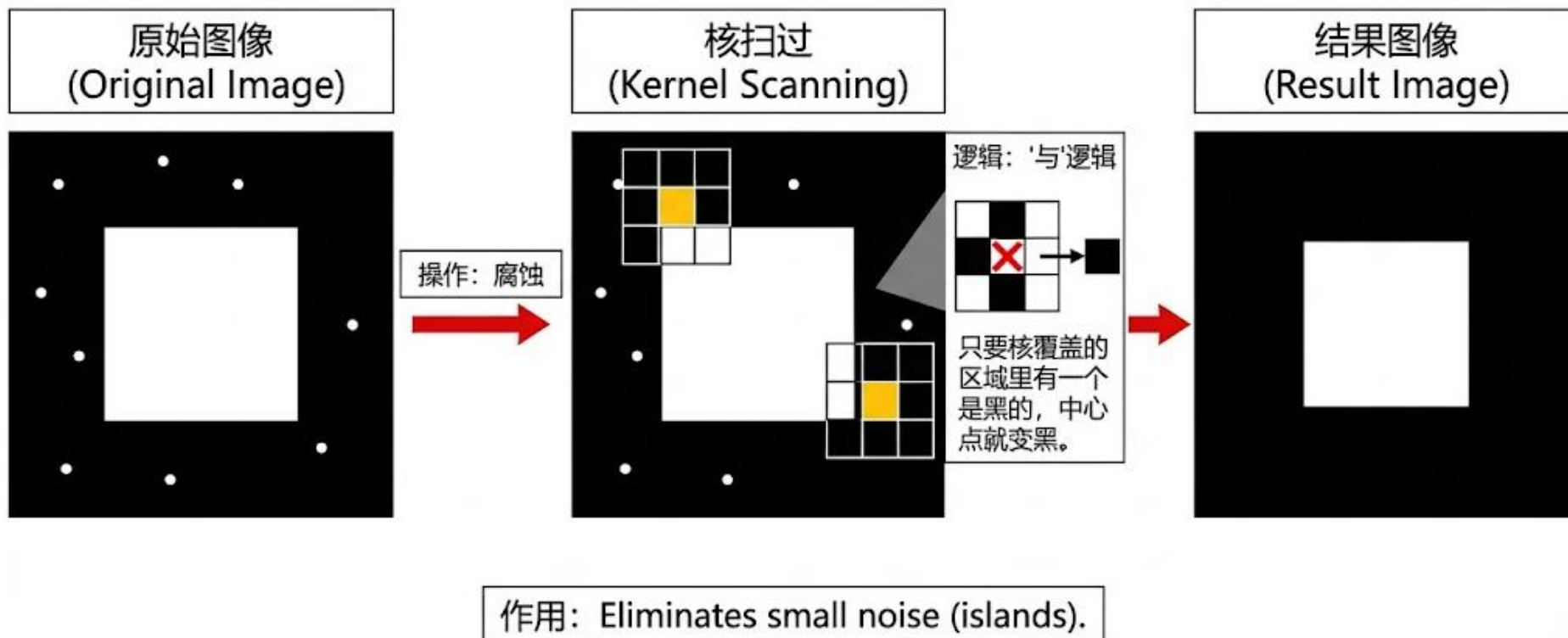
原因分析 (Cause Analysis):

- Sensor Thermal Noise (传感器热噪声)
- Lighting Flicker (光照闪烁)
- Compression Artifacts (压缩伪影)

掩膜与形态学

手术刀之一腐蚀 (Erode)

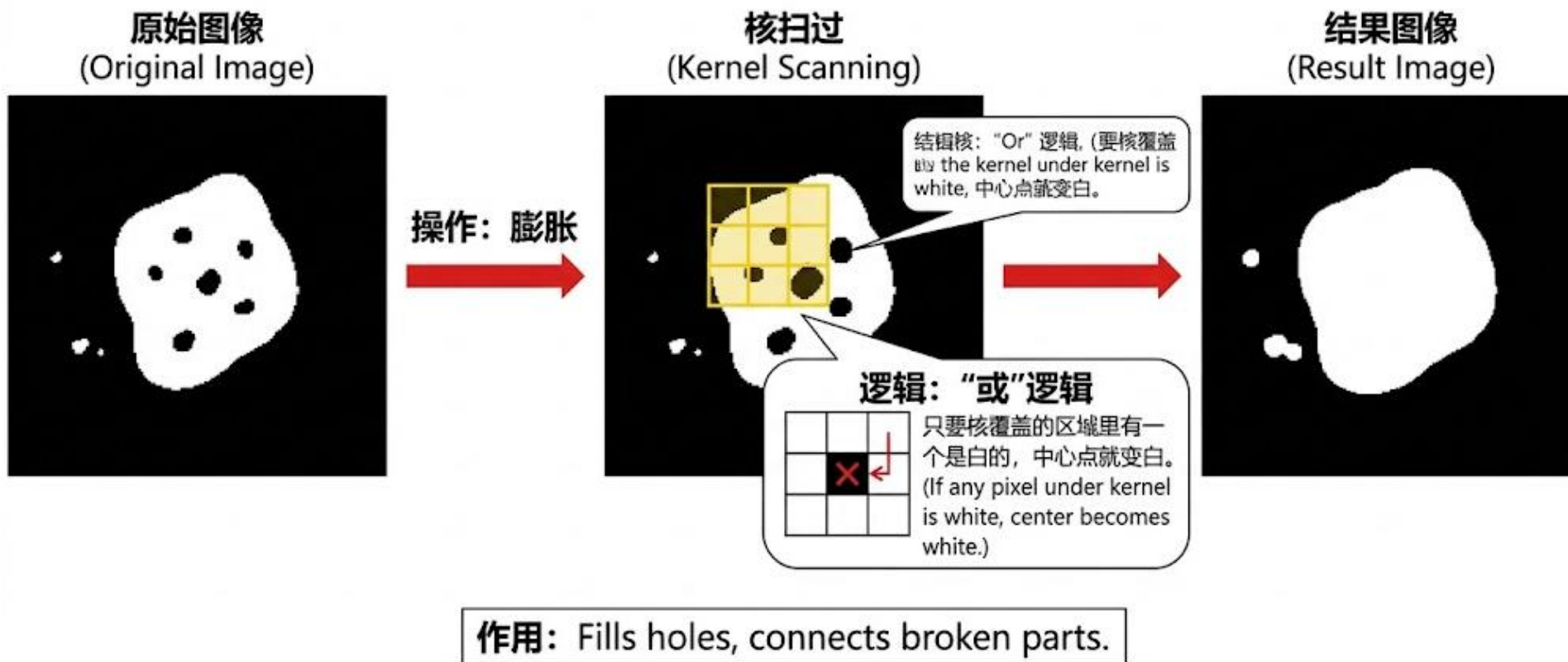
Erosion: Shrinking the Region



掩膜与形态学

手术刀之二—膨胀 (Dilate)

Dilation: Expanding the Region

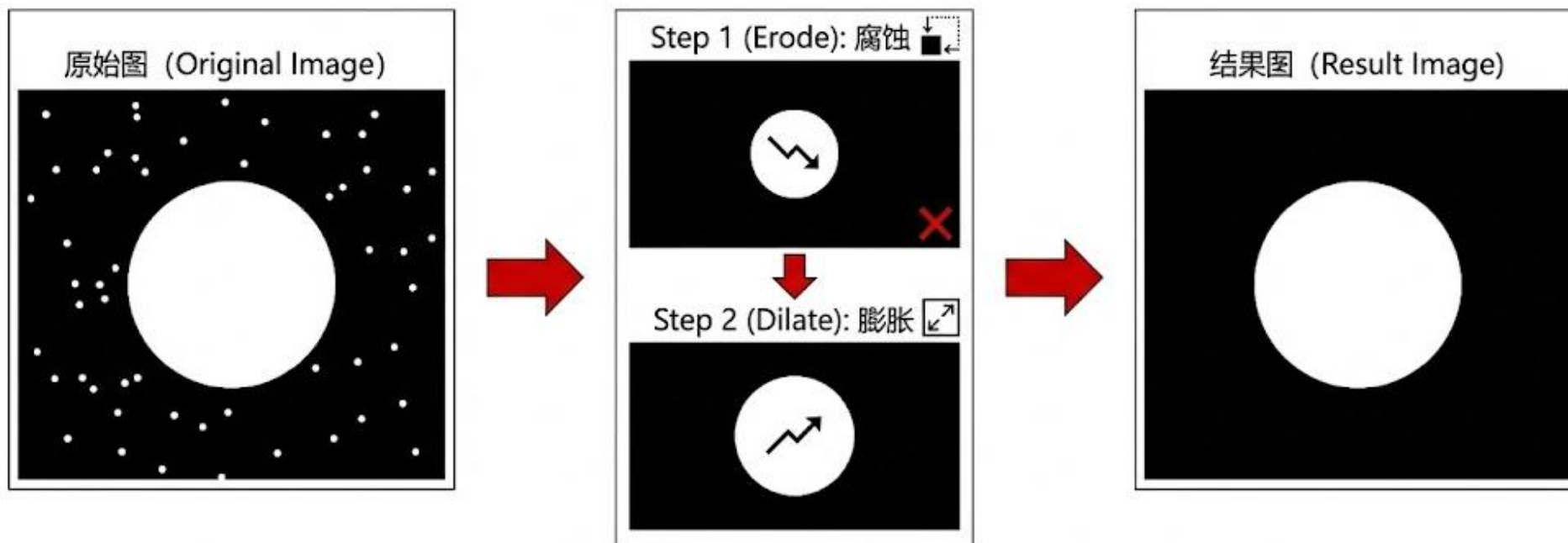


掩膜与形态学

完美的组合拳—开运算 (Opening)

Opening: The Perfect Denoising Combo

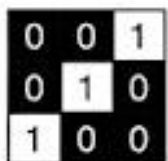
公式: $\text{Opening} = \text{Erosion} \rightarrow \text{Dilation}$



结论: Noise removed, Object size restored.

掩膜与形态学

关键点 (Takeaways)



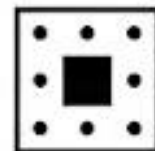
Mask is Binary
(掩膜是二值的 0/1)



Erode (腐蚀): Removes pixels
(Rule: All match)



Dilate (膨胀): Adds pixels
(Rule: Any match)



Open (开运算): Erode then Dilate
(Standard for noise removal)

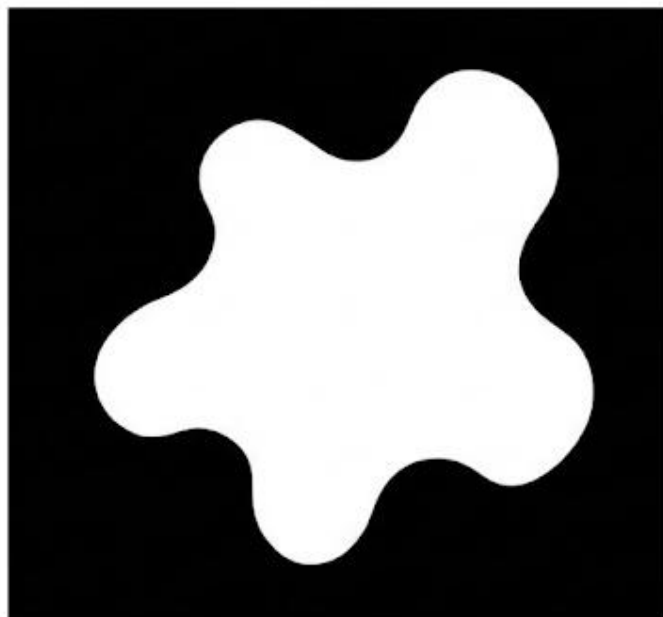
Mask提取器

1. 读取摄像头数据
2. mask操作
3. 去噪
4. 查看结果

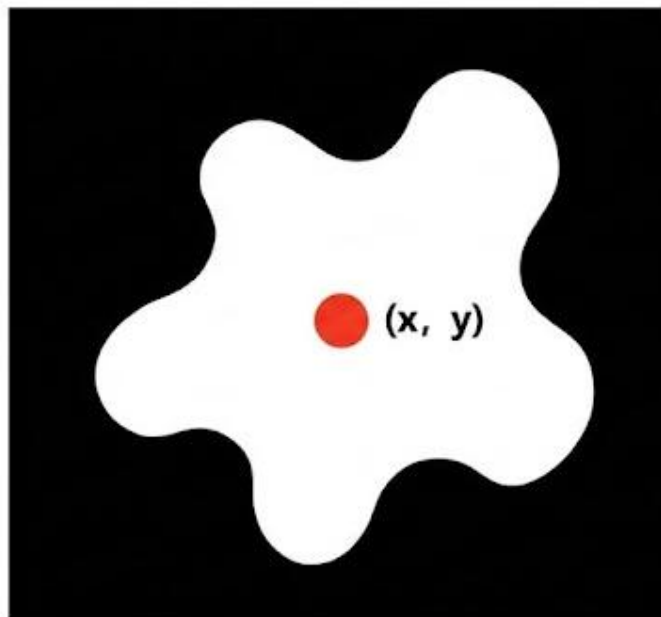
图像几何定位

给机器人一个“靶心”

上节课做好的黑白图（白色的不规则形状）



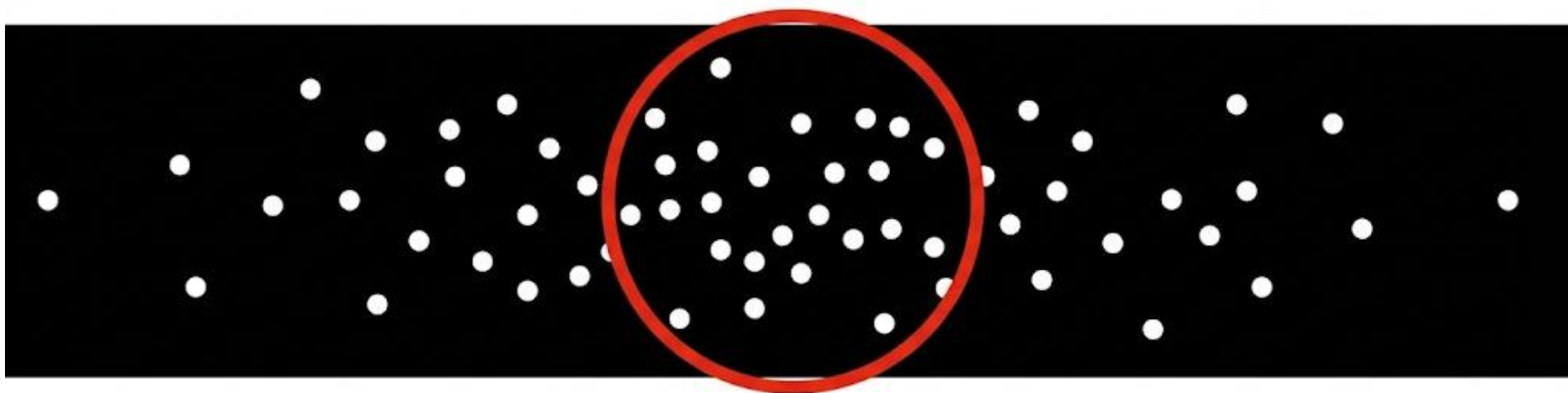
同样的图，但中心多了一个鲜红色的圆点 (x, y)



机器人看不懂“形状”，它只想要一个“坐标”。

图像几何定位

核心原理—求“平均位置”



中心 = 所有白色像素的“平均住址”

类比：就像计算全班同学的平均身高一样简单。

图像几何定位

怎么做？OpenCV 的“自动计数器”

轮廓（那团白色）



cv2.moments()



总和数据

```
{  
  'm00': Area (总像素数),  
  'm10': Sum X (X坐标总和),  
  'm01': Sum Y (Y坐标总和),  
  ...  
}
```

$$X = \frac{\text{X坐标总和}}{\text{总像素数(面积)}}$$

关键点：我们只负责做最后一步除法。

图像几何定位

框选物体，显示坐标

1. 读取摄像头数据
2. 指定hsv阈值
3. 指定腐蚀膨胀阈值
4. 框选出物体
5. 显示物体在图像中的实时坐标



ROS的视觉操作

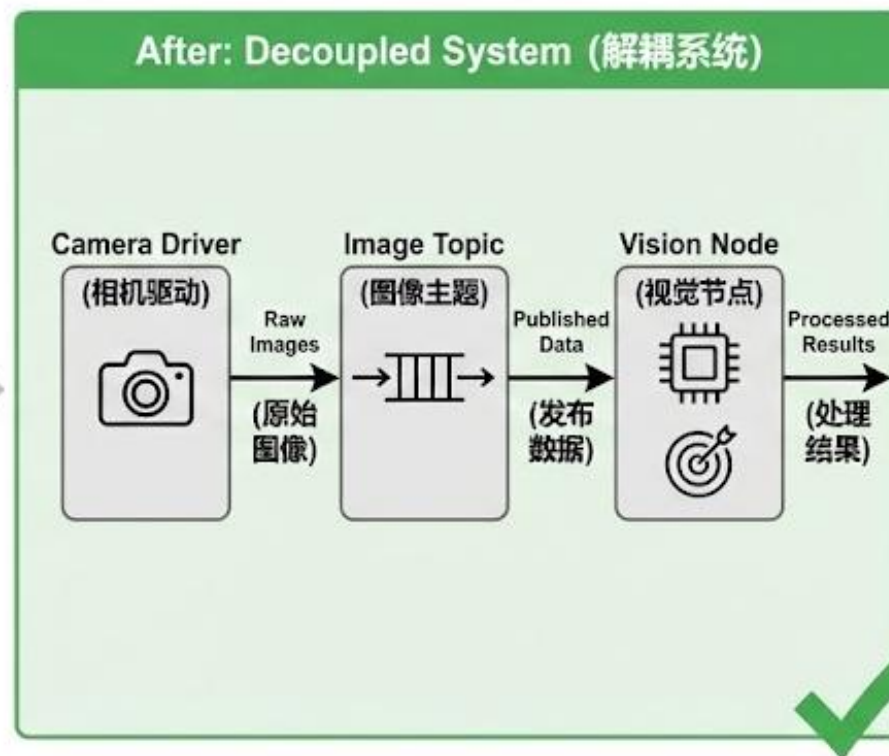
工程思维的转变

Vision Subsystem Engineering



Hard to Maintain, Debug, & Scale
(难以维护、调试和扩展)

Shift in Thinking
(思维转变)



Modular, Reusable, Testable, & Scalable
(模块化、可复用、可测试和可扩展)

摄像头节点

1. 测试虚拟机摄像头

```
sudo apt install cheese
```

2. 配置虚拟机USB3.1

3. 驱动安装

```
sudo apt install ros-jazzy-usb-cam
```

4. 运行摄像头节点

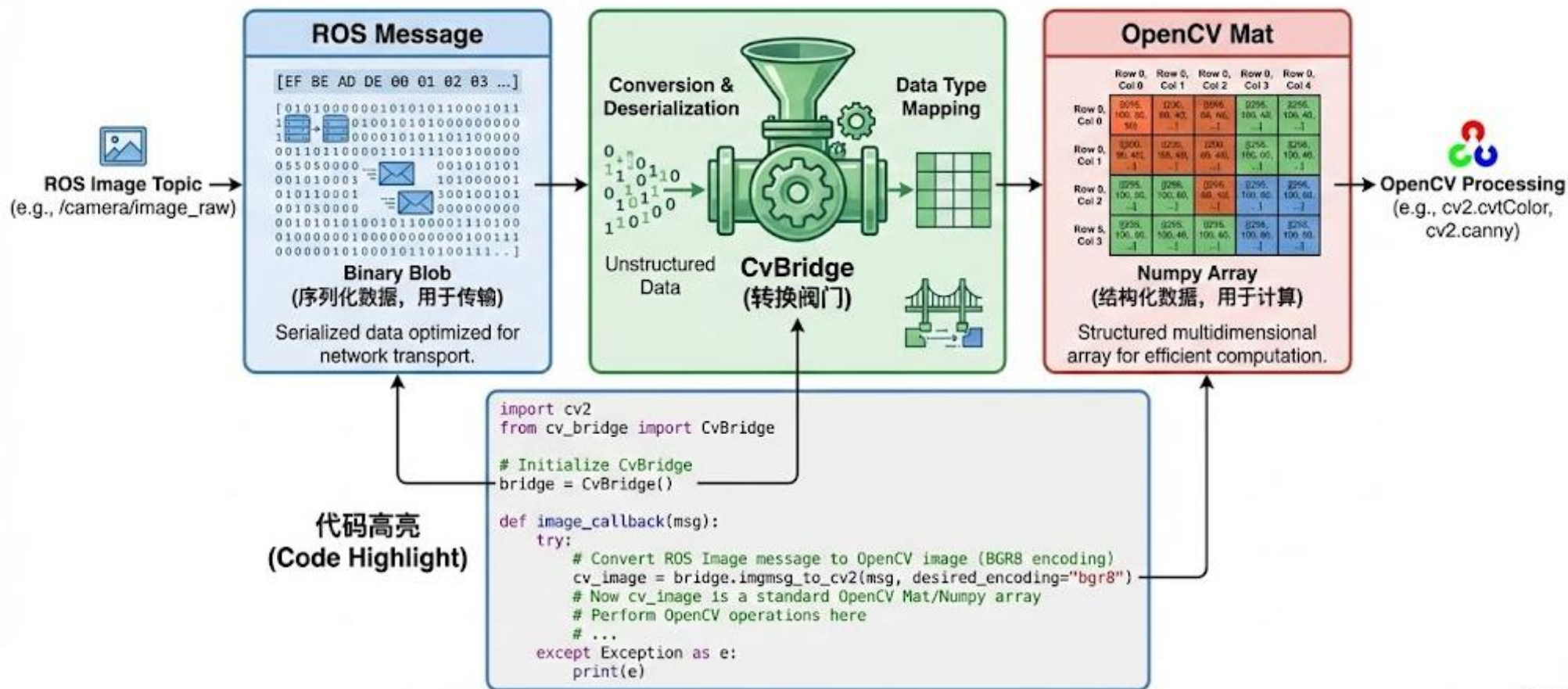
```
ros2 run usb_cam usb_cam_node_exe
```

5. 测试摄像头节点

```
ros2 run rqt_image_view rqt_image_view
```

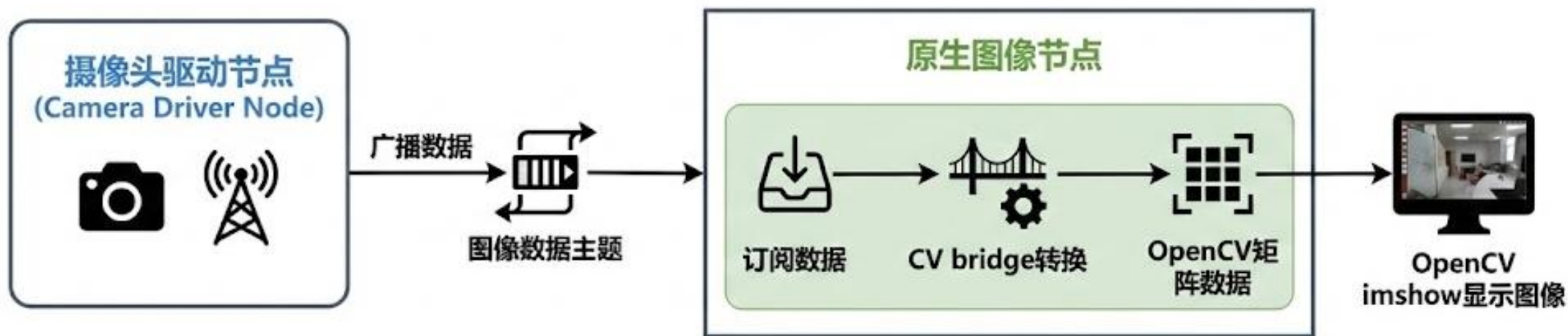
不同内存空间的桥接

Layer 2: The CvBridge Mechanism



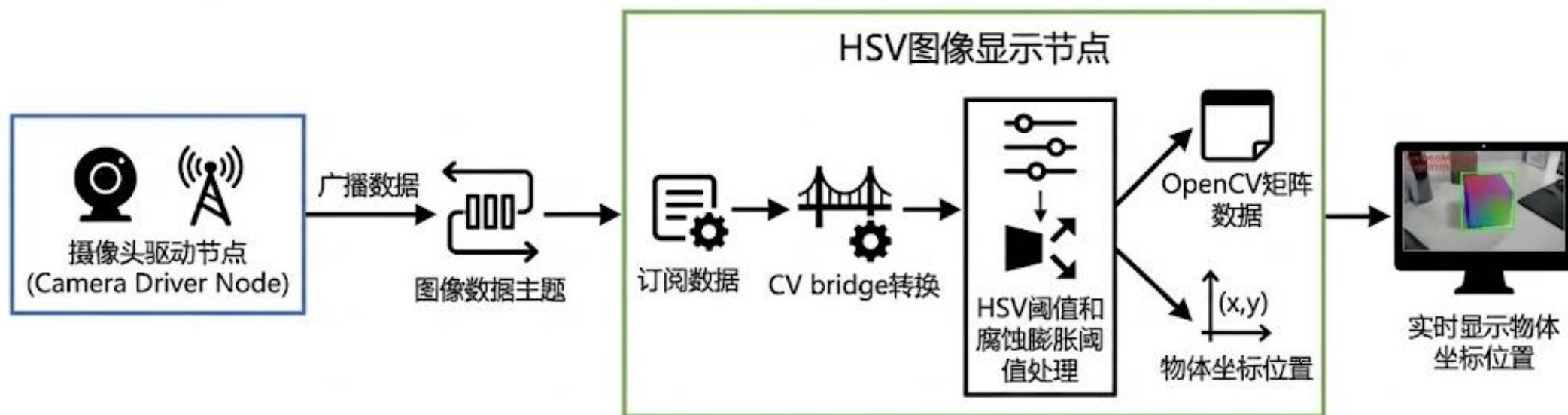
新的需求-Camera显示

原生图像显示节点

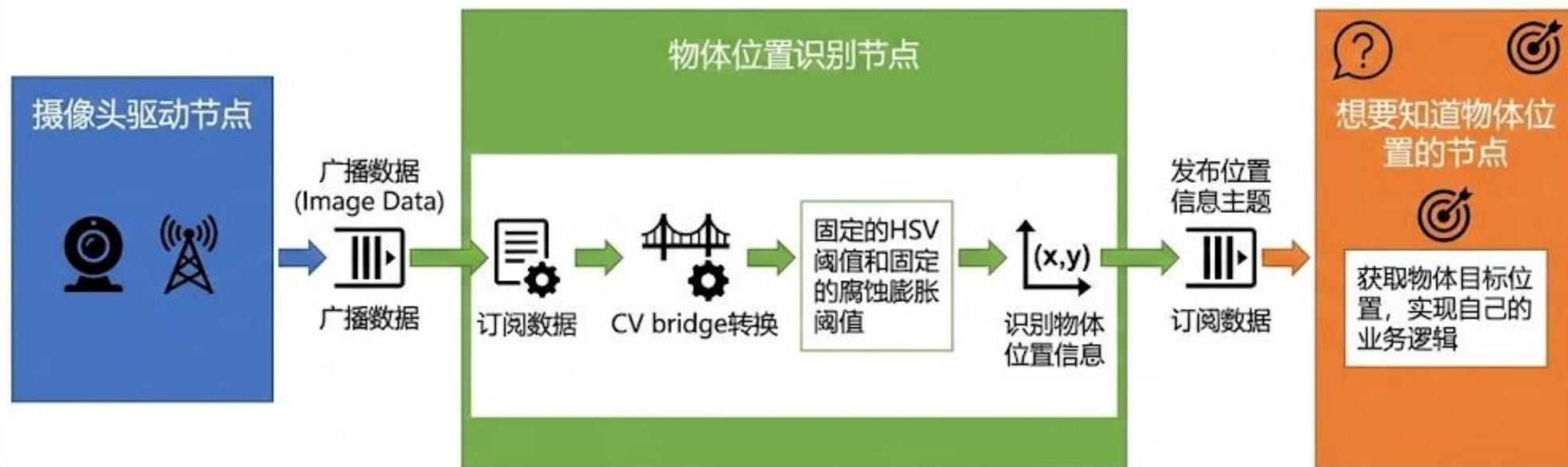


需求提升-HSV框选物体

HSV图像显示节点



需求进阶-物体识别 物体位置识别节点

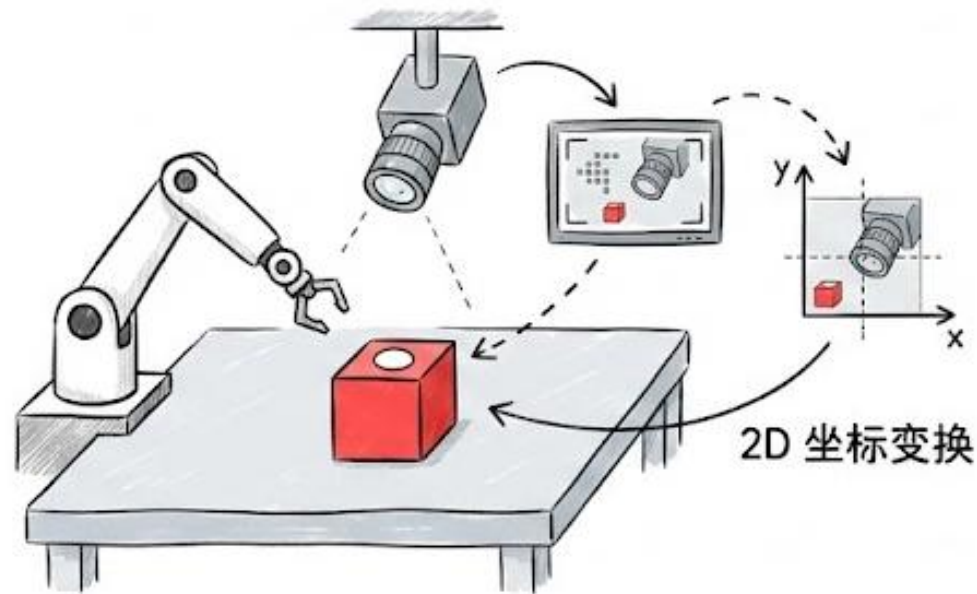




ROS的坐标转换

机械臂的“借眼抓取” (Eye-to-Hand Grasping)

从像素到物理世界的 2D 坐标映射

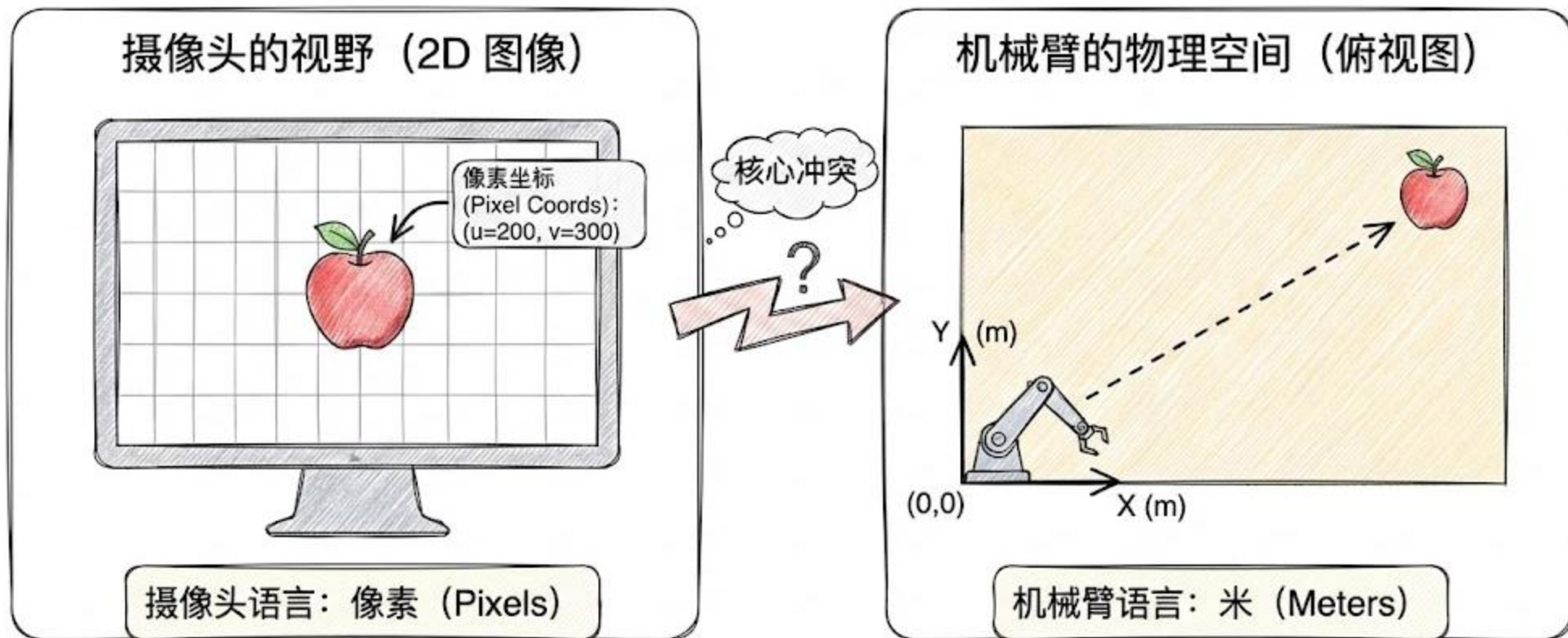


ROS 2 TF2

坐标系

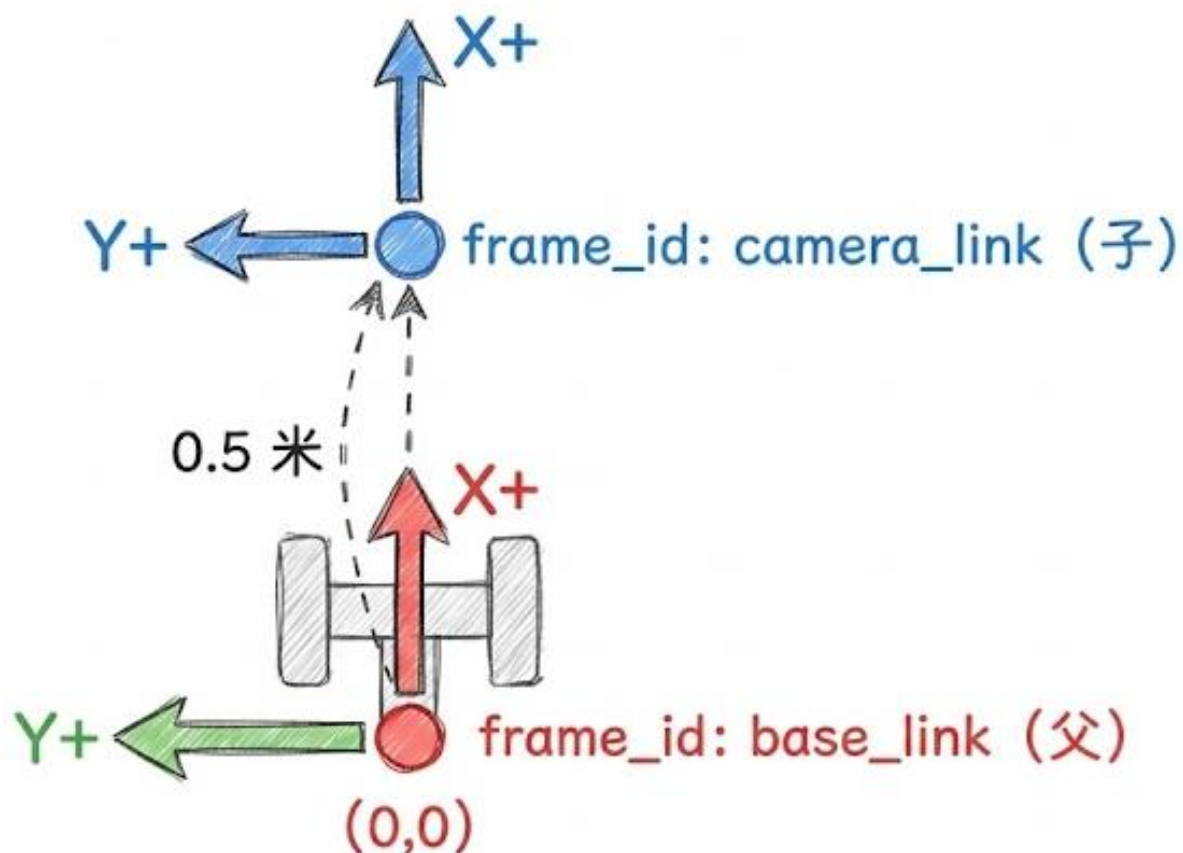
空间变换

场景定义 —— 两个世界的隔阂

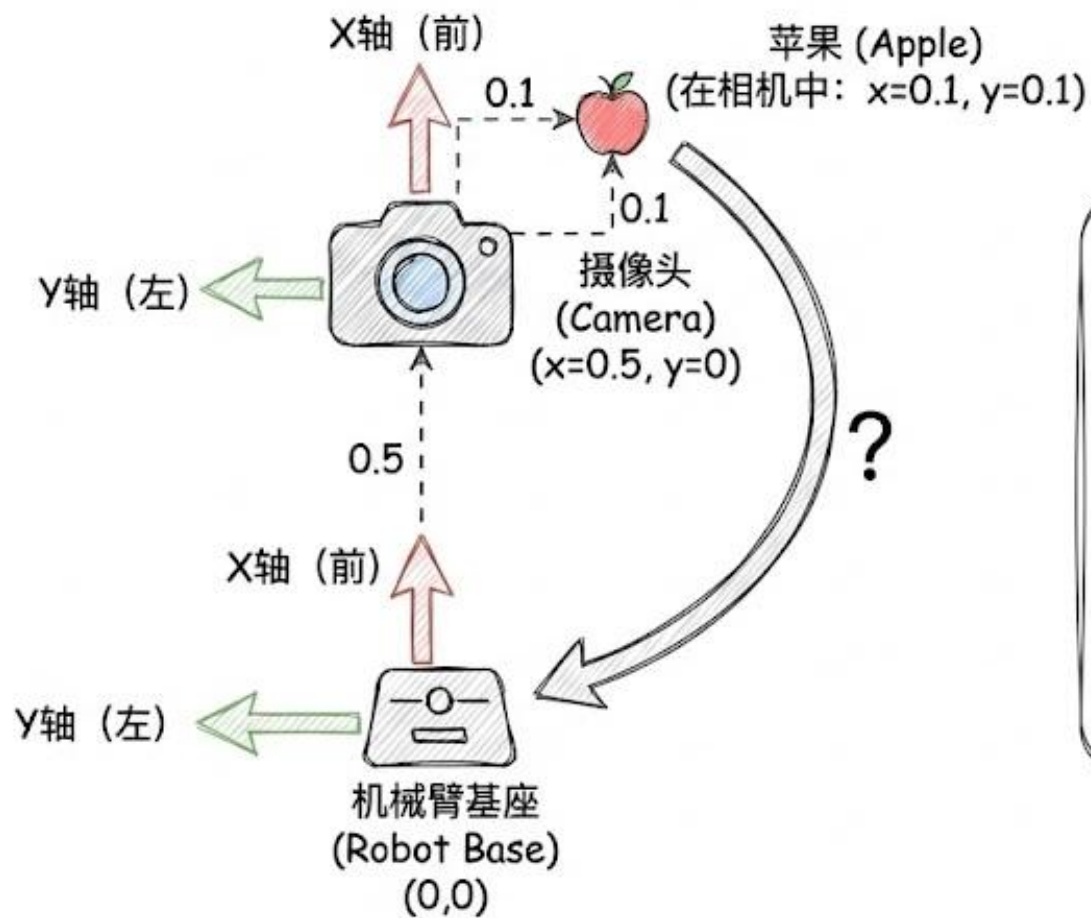


注: 我们忽略高度 (Z), 只看桌面平面 (X, Y)。

坐标系确立 —— 谁是 (0,0) ?



第一层挑战 —— 平移 (Translation)



坐标计算 (Calculation)

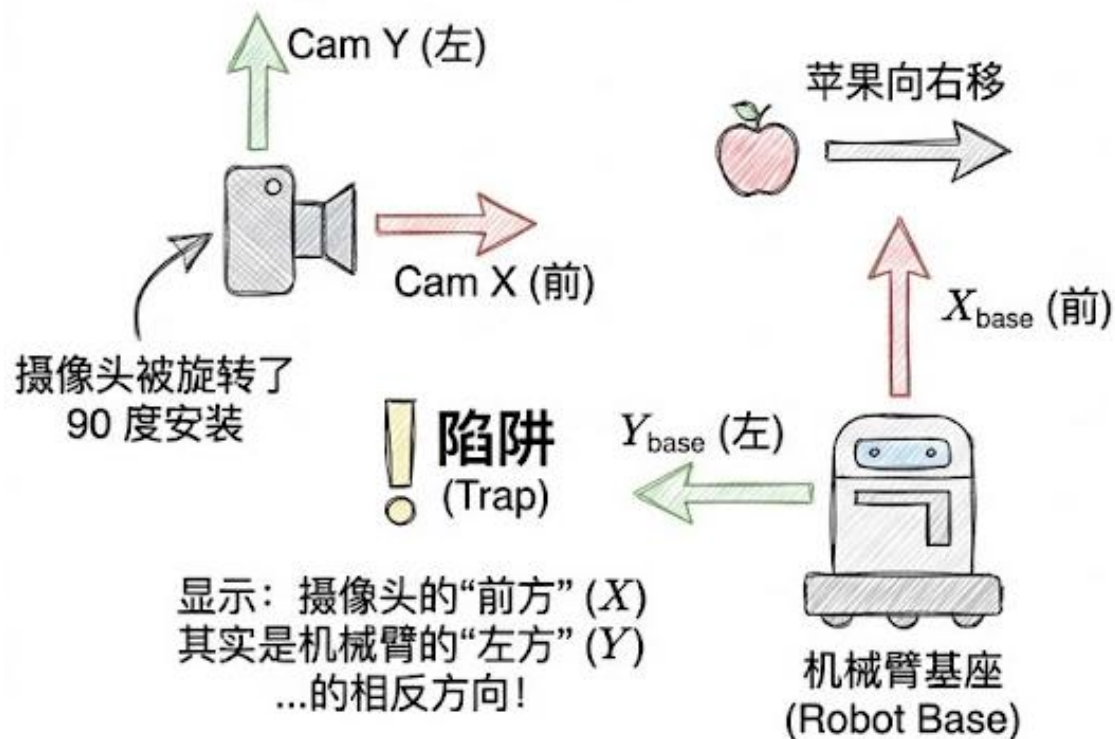
$$X_{final} = 0.5 \text{ (相机位置)} + 0.1 \text{ (苹果在相机里的位置)} \\ = 0.6$$

$$Y_{final} = 0 + 0.1 = 0.1$$

最终坐标 (Final Coordinate): $(0.6, 0.1)$

第二层挑战 —— 旋转 (Rotation) 的陷阱

旋转的陷阱



对比表

动作	摄像头看来	机械臂看来
苹果向右移	X 增加	Y 减小

救世主 —— ROS TF2 工具

```
Node(  
  package='tf2_ros',  
  executable='static_transform_publisher',  
  arguments=['0.5', '0', '0',  
             '1.57',  
             '0', '0',  
             'base_link',  
             'camera_link']  
)
```

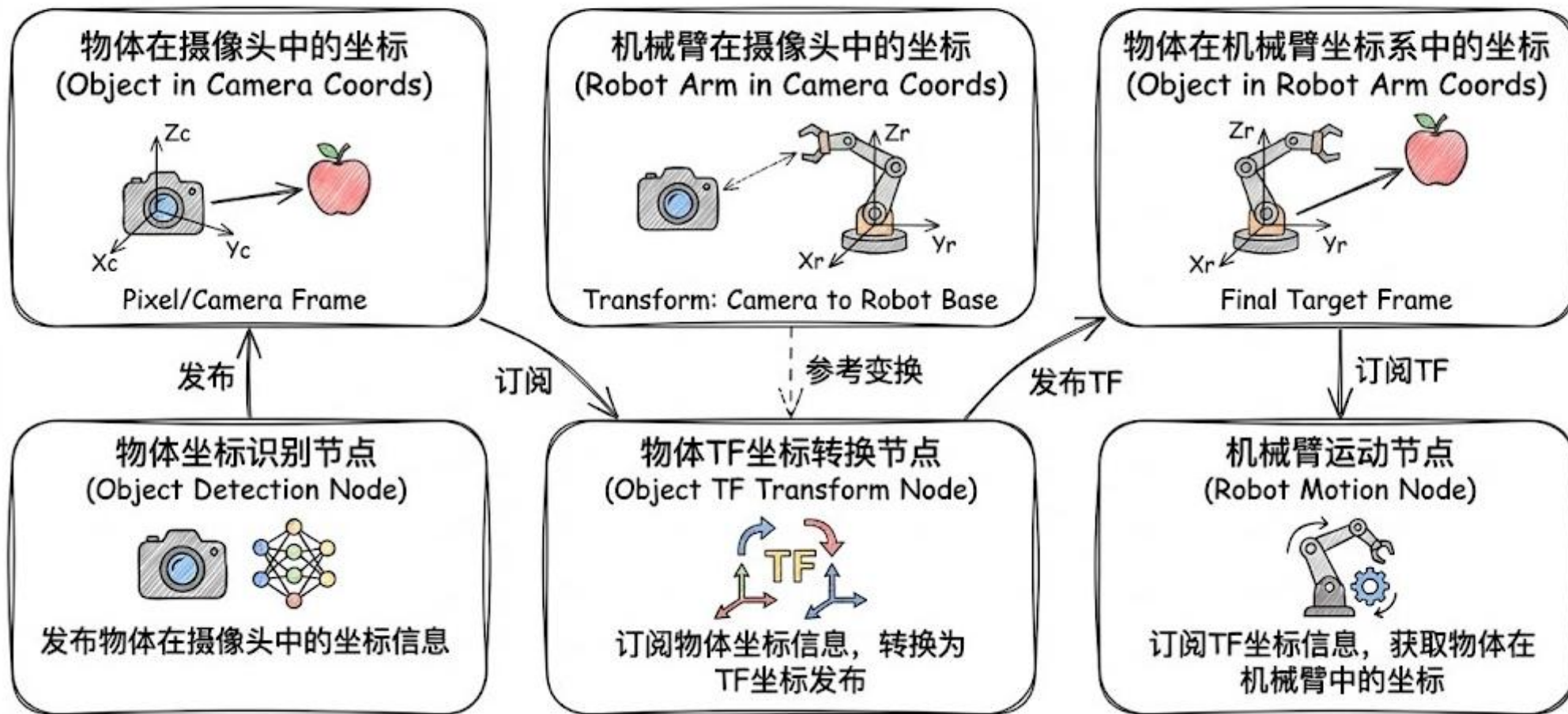
Launch文件

相机看到的坐标

TF 变换

机械臂需要的坐标

实时计算物体在机械臂坐标系中的坐标





正解与反解

什么是运动学? (Kinematics Defined)

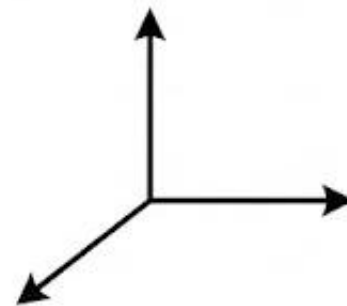
定义：研究物体的运动，不考虑质量和力。



Joint Space
(关节空间 θ)

Forward (FK): $\theta \rightarrow x, y, z$

Inverse (IK): $x, y, z \rightarrow \theta$

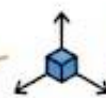


Cartesian Space
(笛卡尔空间 x, y, z)



Forward Kinematics (FK)

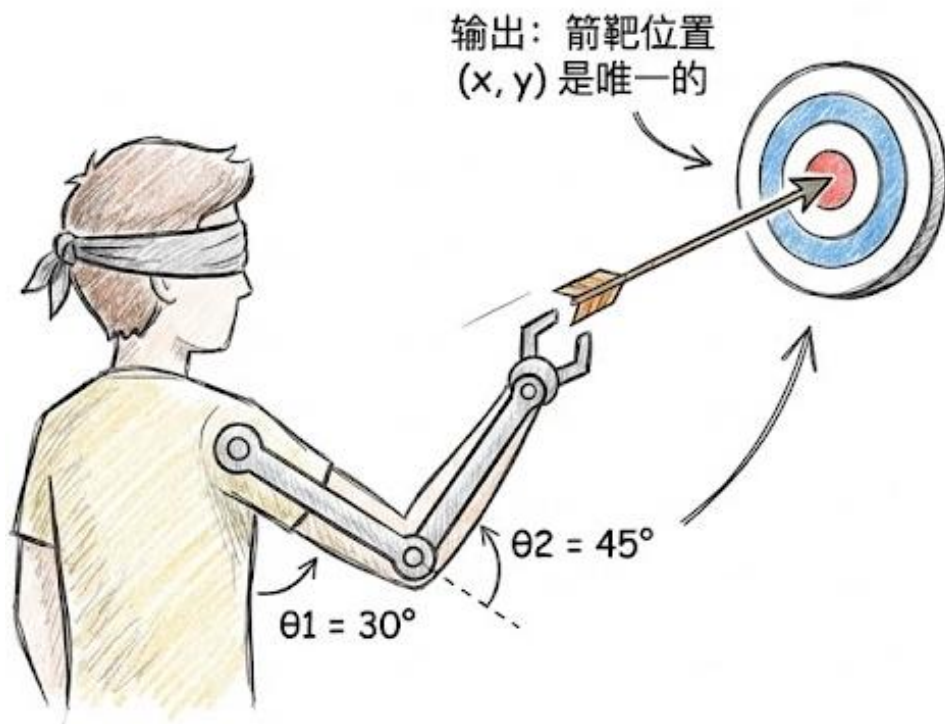
Joint Space (θ) $\rightarrow$ Cartesian Space (x, y, z)



Inverse Kinematics (IK)

Cartesian Space (x, y, z) $\rightarrow$ Joint Space (θ)

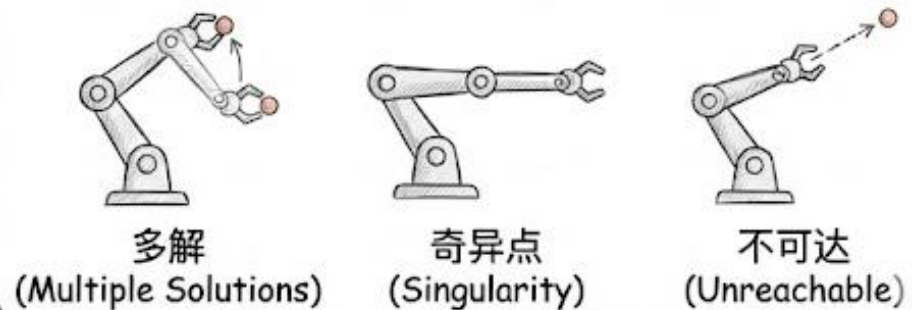
Forward Kinematics (FK): 盲人射箭



Inverse Kinematics (IK): 篮球投手



IK 难点标注



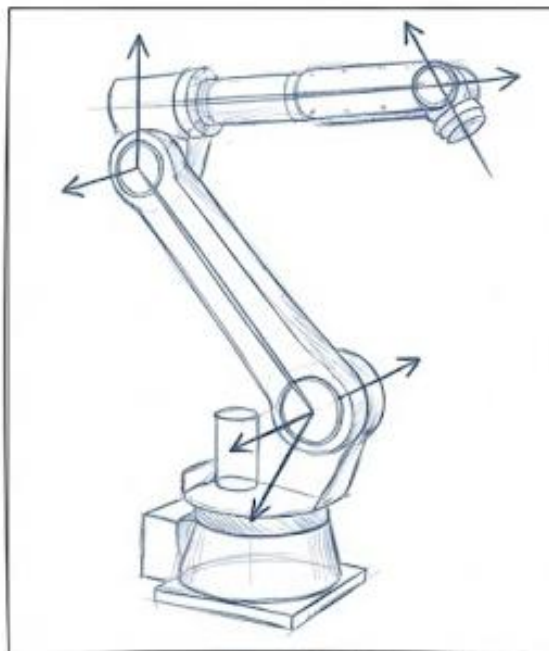


机器人的骨架 (The Skeleton)

Real (真实)



Wireframe (线框图)



Real (真实)

Wireframe (线框图)

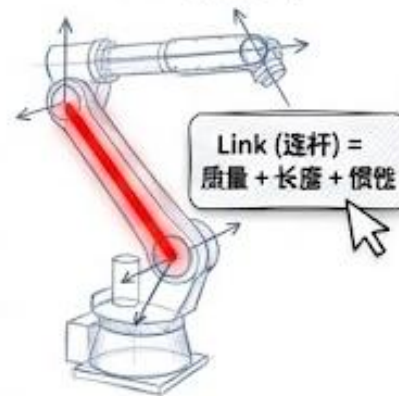
URDF 不是代码，它是机器人的 DNA。

机器人的骨架 (The Skeleton)

Real (真实)



Wireframe (线框图)



机器人的骨架 (The Skeleton)

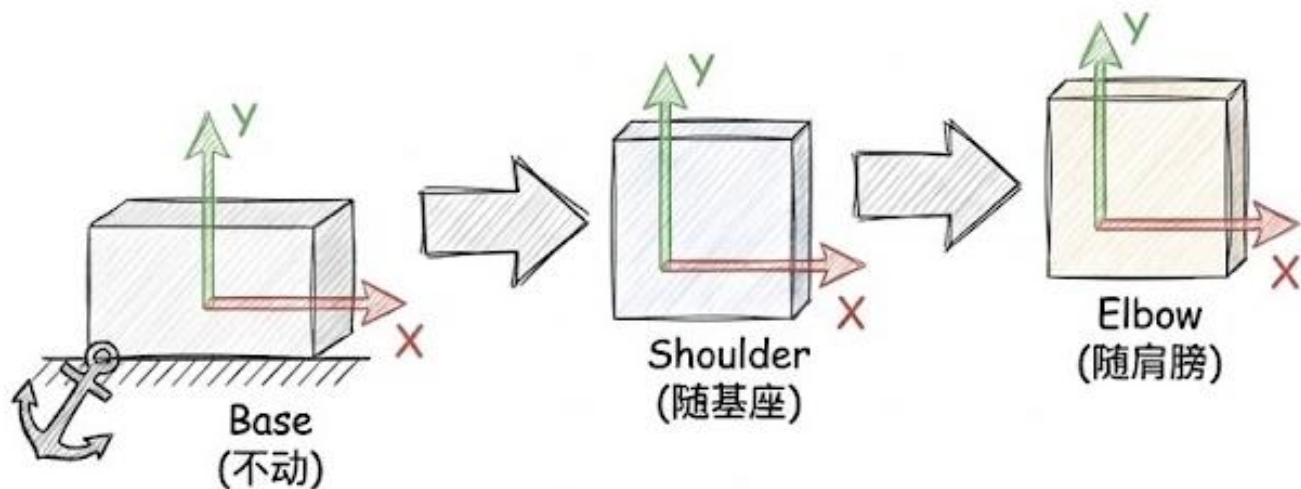
Real (真实)



Wireframe (线框图)

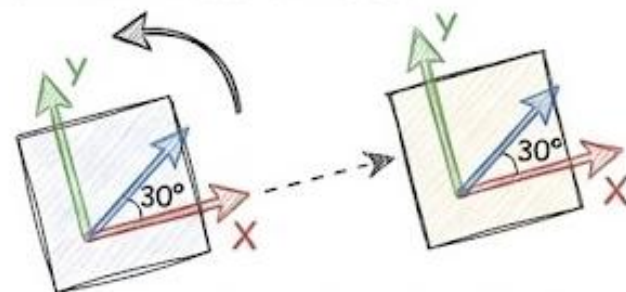


父子关系与变换链 (The Kinematic Chain)



动态演示 (Dynamic View)

当 Shoulder 旋转 30 度时...

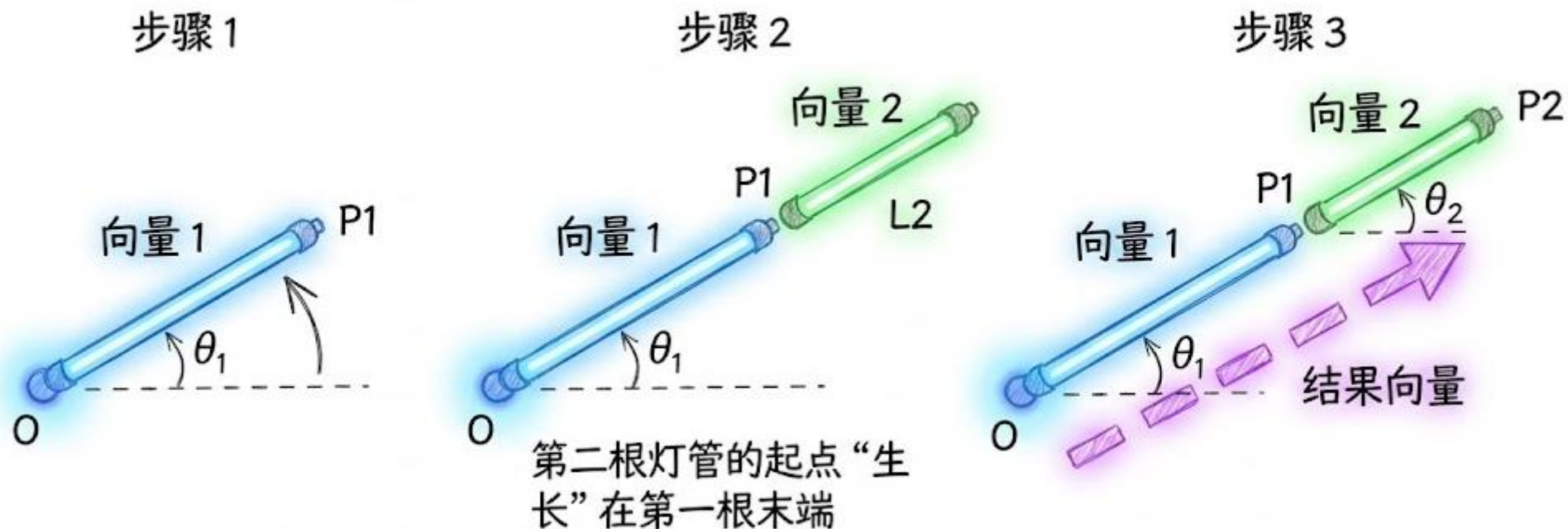


...Elbow 及其后的所有坐标系
整体跟着旋转 30 度。

视觉隐喻：就像一列火车，车头动了，车厢必须跟着动。



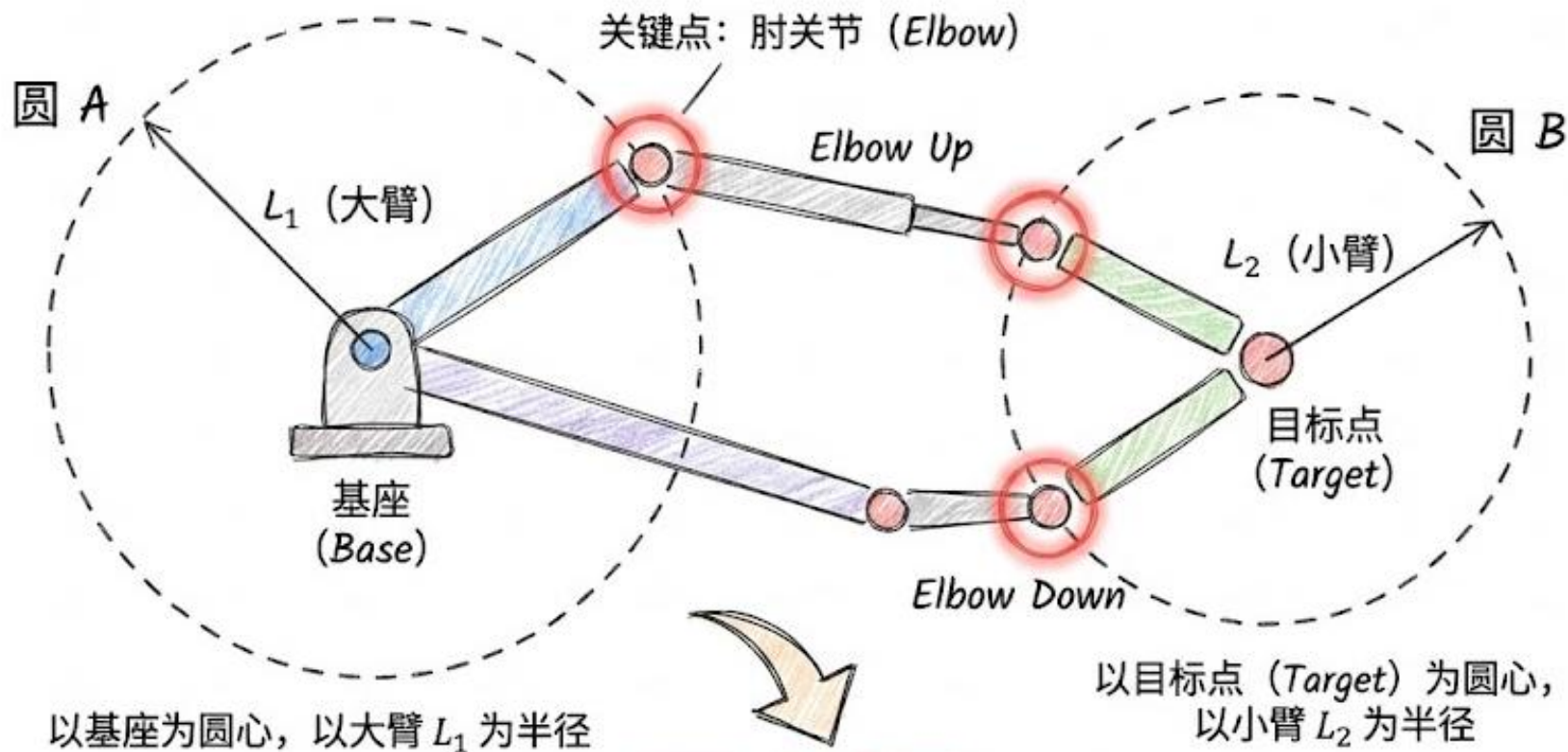
正运动学 —— 向量叠加 (Vector Addition)



结果: 位置 = 向量 1 + 向量 2



逆运动学 —— 几何直觉 (Geometric Intuition)

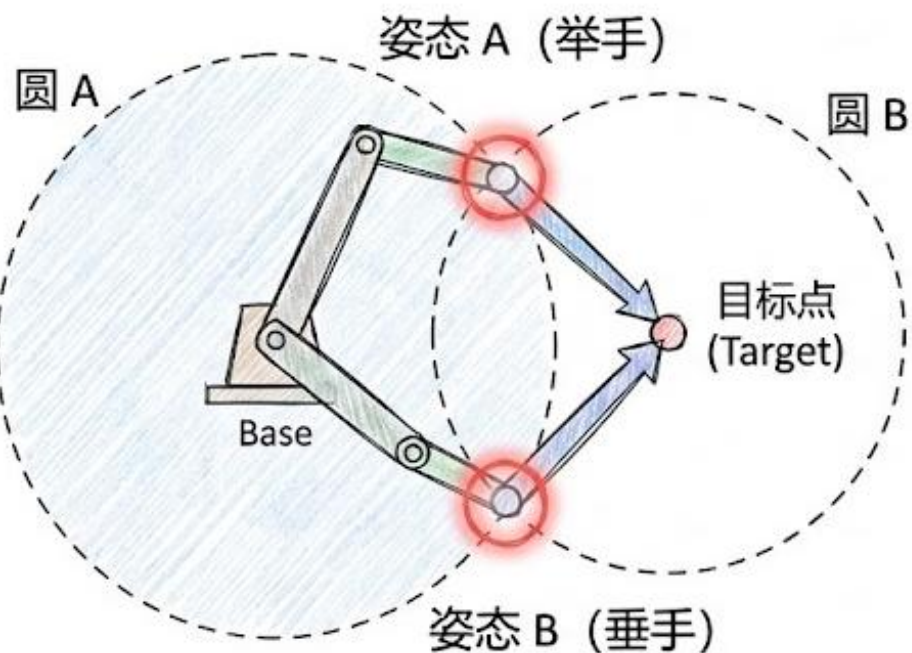


两个解！

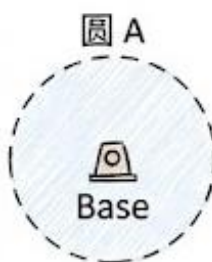
Elbow Up & Elbow Down

多解与奇异点 (Singularity & Solutions)

多解 (Multiple Solutions)

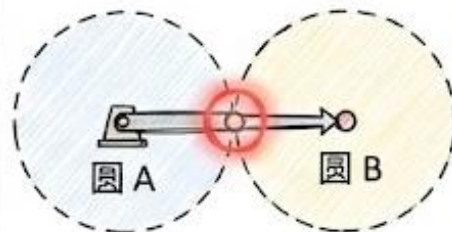


无解/奇异 (No Solution/Singularity)



情况 1: 两个圆相离 (没碰到)

够不着 ✕
(Unreachable)



情况 2: 两个圆相切 (仅一个点)

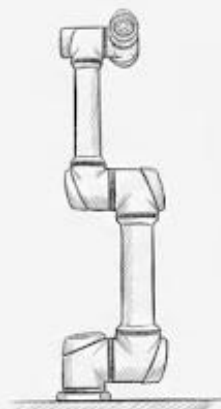
奇异点 (Singularity) ——
手臂伸直锁死，动弹不得。

INPUT (Joint Angles)



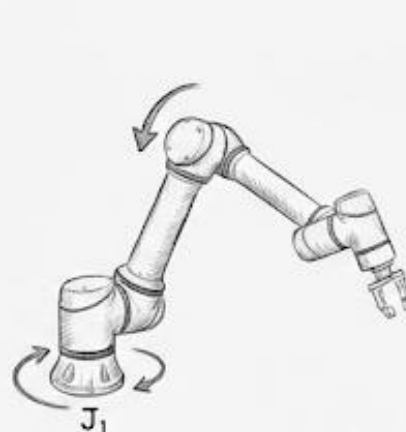
VISUAL PROCESS (3D Robot Arm Snapshots)

Snapshot 1 (Home)



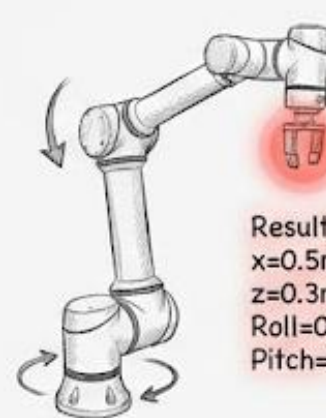
Home Position
(All 0°)

Snapshot 2 (Action)



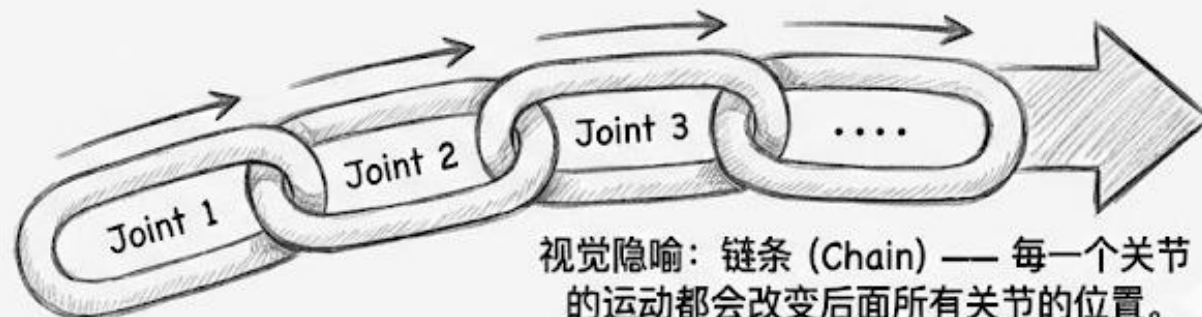
Action
(Base Rotate, Shoulder Lower)

Snapshot 3 (Result)



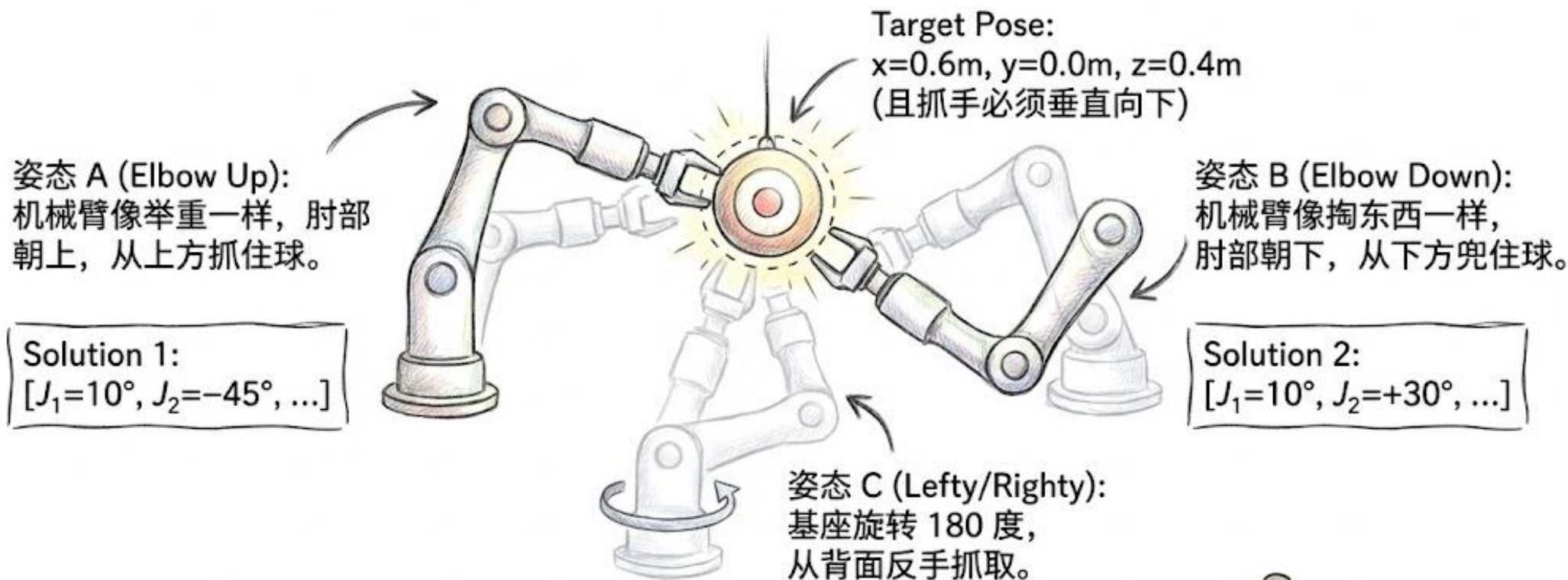
Final Result
(Elbow Lift, Gate Shape)

Result Pose:
 $x=0.5m, y=0.5m,$
 $z=0.3m,$
Roll=0,
Pitch= 90°

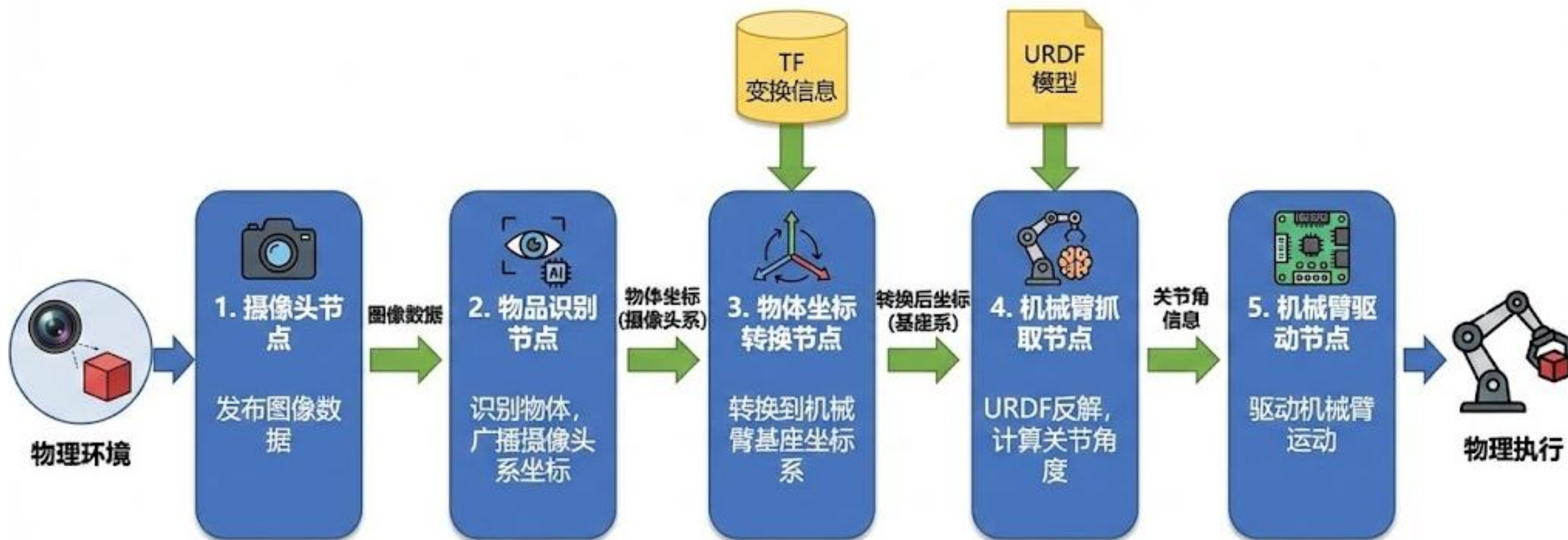


视觉隐喻：链条 (Chain) —— 每一个关节的运动都会改变后面所有关节的位置。

条条马路通罗马 (IK Case Study)



Total Solutions: n组解 (对于标准的 6 轴机械臂)。





传智教育旗下高端IT教育品牌